UNIT- IV
FILE HANDLING

WHAT IS FILE?

File is a collection of bytes that is stored on secondary storage devices like disk. There are two kinds of files in a system. They are,

1. Text files (ASCII)
2. Binary files

- Text files contain ASCII codes of digits, alphabetic and symbols.
- Binary file contains collection of bytes (0"s and 1"s). Binary files are compiled version of text files.

BASIC FILE OPERATIONS IN C PROGRAMMING:

There are 4 basic operations that can be performed on any files in C programming language. They are,

Different operations that can be performed on a file are:

1. Creation of a new file (**fopen with attributes as "a" or "a+" or "w" or "w++")**
2. Opening an existing file (**fopen**)
3. Reading from file (**fscanf or fgetc**)
4. Writing to a file (**fprintf or fputs**)
5. Moving to a specific location in a file (**fseek, rewind**)
6. Closing a file (**fclose**)

As given above, if you want to perform operations on a binary file, then you have to append „b" at the last. For example, instead of "w", you have to use "wb", instead of "a+" you have to use "a+b". For performing the operations on the file, a special pointer called File pointer is used which is declared as

```
FILE *filePointer;

So, the file can be opened as

filePointer = fopen("fileName.txt", "w")
```

The second parameter can be changed to contain all the attributes listed in the above table.

- **Reading from a file –**
  The file read operations can be performed using functions fscanf or fgets. Both the functions performed the same operations as that of scanf and gets but with an additional parameter, the file pointer. So, it depends on you if you want to read the file line by line or character by character.
  And the code snippet for reading a file is as:

```
FILE * filePointer;

filePointer = fopen("fileName.txt", "r");

fscanf(filePointer, "%s %s %s %d", str1, str2, str3, &year);
```

- **Writing a file –**:
  The file write operations can be perfomed by the functions fprintf and fputs with similarities to read operations. The snippet for writing to a file is as :

```
FILE *filePointer ;

filePointer = fopen(“fileName.txt”, “w”);

fprintf(filePointer, "%s %s %s %d", "We", "are", "in", 2012);
```

- **Closing a file –**:
  After every successful fie operations, you must always close a file. For closing a file, you have to use fclose function. The snippet for closing a file is given as :

```
• FILE *filePointer ;
• filePointer= fopen(“fileName.txt”, “w”);
• ---------- Some file Operations -------
```

```
fclose(filePointer)
```

:

MODE OF OPERATIONS PERFORMED ON A FILE IN C LANGUAGE:
There are many modes in opening a file. Based on the mode of file, it can be opened for reading or writing or appending the texts. They are listed below.

- r – Opens a file in read mode and sets pointer to the first character in the file. It returns null if file does not exist.
- w – Opens a file in write mode. It returns null if file could not be opened. If file exists, data are overwritten.
- a – Opens a file in append mode. It returns null if file couldn‟t be opened.
- r+ – Opens a file for read and write mode and sets pointer to the first character in the file.
- w+ – opens a file for read and write mode and sets pointer to the first character in the file.
- a+ – Opens a file for read and write mode and sets pointer to the first character in the file. But, it can‟t modify existing contents.

INBUILT FUNCTIONS FOR FILE HANDLING IN C LANGUAGE:
C programming language offers many inbuilt functions for handling files. They are given below. Please click on each function name below to know more details, example programs, output for the respective file handling function.

File handling functions	Description
fopen ()	fopen () function creates a new file or opens an existing file.

fclose ()	fclose () function closes an opened file.
getw ()	getw () function reads an integer from file.
putw ()	putw () functions writes an integer to file.
fgetc ()	fgetc () function reads a character from file.
fputc ()	fputc () functions write a character to file.
gets ()	gets () function reads line from keyboard.
puts ()	puts () function writes line to o/p screen.
fgets ()	fgets () function reads string from a file, one line at a time.
fputs ()	fputs () function writes string to a file.
feof ()	feof () function finds end of file.
fgetchar ()	fgetchar () function reads a character from keyboard.
fprintf ()	fprintf () function writes formatted data to a file.
fscanf ()	fscanf () function reads formatted data from a file.
fputchar ()	fputchar () function writes a character onto the output screen from keyboard input.
fseek ()	fseek () function moves file pointer position to given location.

SEEK_SET	SEEK_SET moves file pointer position to the beginning of the file.
SEEK_CUR	SEEK_CUR moves file pointer position to given location.
SEEK_END	SEEK_END moves file pointer position to the end of file.
ftell ()	ftell () function gives current position of file pointer.
rewind ()	rewind () function moves file pointer position to the beginning of the file.
getc ()	getc () function reads character from file.
getch ()	getch () function reads character from keyboard.
getche ()	getche () function reads character from keyboard and echoes to o/p screen.
getchar ()	getchar () function reads character from keyboard.
putc ()	putc () function writes a character to file.
putchar ()	putchar () function writes a character to screen.
printf ()	printf () function writes formatted data to screen.
sprinf ()	sprinf () function writes formatted output to string.

scanf ()	scanf () function reads formatted data from keyboard.
sscanf ()	sscanf () function Reads formatted input from a string.
remove ()	remove () function deletes a file.
fflush ()	fflush () function flushes a file.

Example 1: Program to Open a File, Write in it, And Close the File

```
// C program to Open a File,
// Write in it, And Close the File

# include <stdio.h>
# include <string.h>

int main( )
{

    // Declare the file pointer
    FILE *filePointer ;

    // Get the data to be written in file
    char dataToBeWritten[50]
        = "GeeksforGeeks-A Computer Science Portal for Geeks";

    // Open the existing file GfgTest.c using fopen()
    // in write mode using "w" attribute
    filePointer = fopen("GfgTest.c", "w") ;

    // Check if this filePointer is null
    // which maybe if the file does not exist
    if ( filePointer == NULL )
    {
        printf( "GfgTest.c file failed to open." ) ;
    }
    else
    {

        printf("The file is now opened.\n") ;

        // Write the dataToBeWritten into the file
        if ( strlen ( dataToBeWritten ) > 0 )
        {

            // writing in the file using fputs()
            fputs(dataToBeWritten, filePointer) ;
```

```
            fputs("\n", filePointer) ;
        }

        // Closing the file using fclose()
        fclose(filePointer) ;

        printf("Data successfully written in file GfgTest.c\n");
        printf("The file is now closed.") ;
    }
    return 0;
}
```

Example 2: Program to Open a File, Read from it, And Close the File

```
// C program to Open a File,
// Read from it, And Close the File

# include <stdio.h>
# include <string.h>

int main( )
{

    // Declare the file pointer
    FILE *filePointer ;

    // Declare the variable for the data to be read from file
    char dataToBeRead[50];

    // Open the existing file GfgTest.c using fopen()
    // in read mode using "r" attribute
    filePointer = fopen("GfgTest.c", "r") ;

    // Check if this filePointer is null
    // which maybe if the file does not exist
    if ( filePointer == NULL )
    {
        printf( "GfgTest.c file failed to open." ) ;
    }
    else
    {

        printf("The file is now opened.\n") ;

        // Read the dataToBeRead from the file
        // using fgets() method
        while( fgets ( dataToBeRead, 50, filePointer ) != NULL )
        {

            // Print the dataToBeRead
            printf( "%s" , dataToBeRead ) ;
         }

        // Closing the file using fclose()
        fclose(filePointer) ;
```

```
        printf("Data successfully read from file GfgTest.c\n");
        printf("The file is now closed.") ;
    }
    return 0;
}
```

https://www.w3schools.in/c-tutorial/file-handling/

https://www.studytonight.com/c/file-input-output.php